

Exp No:2**Implementation of a Calculator that takes an expression (with digit + & *) compute and print its value using YACC tool****Aim:**

To implement a simple calculator using **YACC** that accepts an arithmetic expression containing **digits, addition (+) and multiplication (*) operators**, evaluates the expression according to operator precedence, and displays the result.

Algorithm

1. Start the program.
2. Create a Lex file to recognize digits, +, *, and newline.
3. Create a YACC file containing grammar rules for the arithmetic expression.
4. Define the precedence of operators, with * having higher precedence than +.
5. Save the Lex file as calc.l and the YACC file as calc.y.
6. Generate the Lex C source file using flex calc.l.
7. Generate the YACC parser using bison -dy calc.y.
8. Compile the generated C files using GCC.
9. Execute the generated calculator program.
10. Enter an arithmetic expression containing digits, +, and *.
11. YACC parses the expression and calculates its value.
12. Display the calculated result.
13. Stop the program.

Lex Program — calc.l :

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}

%%

[0-9]+ { yylval = atoi(yytext); return NUMBER; }
"+"    { return '+'; }
"*"    { return '*'; }
"\n"   { return '\n'; }
[ \t]  { }

.      { return yytext[0]; }

%%

int yywrap()
{
    return 1;
}
```

YACC Program — calc.y :

```
%{
#include <stdio.h>
```

```

#include <stdlib.h>

int yylex();
int yyerror(char *s);
%}

%token NUMBER

%left '+'
%left '*'

%%

input:
    expr '\n'    { printf("Result = %d\n", $1); }
    ;

expr:
    expr '+' expr    { $$ = $1 + $3; }
  | expr '*' expr    { $$ = $1 * $3; }
  | NUMBER           { $$ = $1; }
    ;

%%

int yyerror(char *s)
{
    printf("Invalid expression\n");
    return 0;
}

int main()
{
    printf("Enter an expression: ");
    yyparse();
    return 0;
}

```

Procedure to Execute

Step 1: Create the Lex File

Open **Notepad** and paste the Lex program given above.

Save it as:

calc.l

While saving, select:

Save as type → All Files

Make sure it is saved as:

calc.l

and **not** calc.l.txt.

Step 2: Create the YACC File

Open another **Notepad** window and paste the YACC program given above.

Save it as:

calc.y

Again, select:

Save as type → All Files

Make sure it is:

calc.y

Step 3: Open Command Prompt

Open **Command Prompt** and move to the folder where you saved both files.

For example:

cd Desktop

Check the files:

ls

You should see:

calc.l

calc.y

Step 4: Generate Lexical Analyzer

Type:

flex calc.l

If there is no error, the following file will be generated:

lex.yy.c

Step 5: Generate YACC Parser

Type:

bison -dy calc.y

If there is no error, the following files will be generated:

y.tab.c

y.tab.h

Now your folder should contain:

calc.l

calc.y

lex.yy.c

y.tab.c

y.tab.h

Step 6: Compile the Generated Files

Type:

gcc lex.yy.c y.tab.c -o calc.exe

If there is no error, the executable file:

calc.exe

will be created.

Step 7: Run the Calculator

Type:

calc.exe

The program will display:

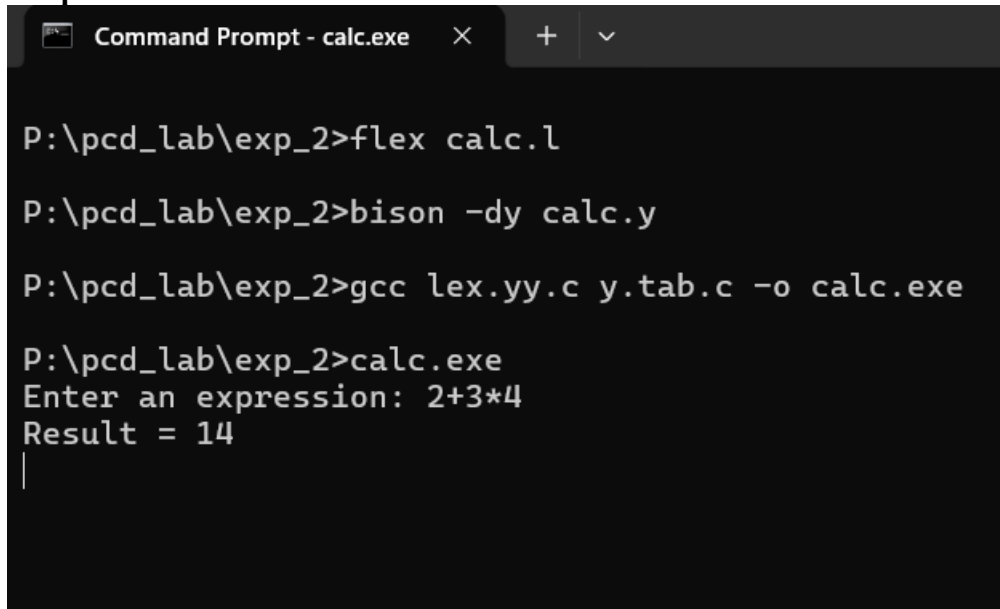
Enter an expression:

Enter:

2+3*4

Press **Enter**.

Output



```
Command Prompt - calc.exe
P:\pcd_lab\exp_2>flex calc.l
P:\pcd_lab\exp_2>bison -dy calc.y
P:\pcd_lab\exp_2>gcc lex.yy.c y.tab.c -o calc.exe
P:\pcd_lab\exp_2>calc.exe
Enter an expression: 2+3*4
Result = 14
|
```

Result

Thus, a simple calculator was successfully implemented using **LEX and YACC**, which accepts arithmetic expressions containing digits, +, and *, evaluates them according to operator precedence, and displays the result.